

Four-Bit Full Adder Using an FPGA

CENG 2112.01 Lab for Digital Circuits

Submitted by

David Dulaney

2344357

Computer Science

University of Houston-Clear Lake

Houston, Texas 77058

Mar 31,2026

Contents

Abstract	2
Introduction	2
Requirements	3
Prelab	4
Implementation	6
Design Code/Design Diagrams	6
Schematics	9
Testbench	10
Waveform/Results	14
Conclusion	23

Abstract

A full adder is a combinational circuit that requires three inputs and will produce two outputs. The first two inputs are normal inputs (A and B), the third input is a carry (Cin). The normal output is the SUM, which is the normal sum of A and B. The other output (Cout) is a carry output which transfers over if a 0 or a 1 is being carried over. For example $1 + 1 + 1$ will be a SUM of 1 and have a Cout as 1.

Introduction

A full adder (#7483) is a device that when given three inputs, it will produce two outputs, with one being the sum of the inputs and the other being the carry value of the inputs (indicated by a 0 or 1). The purpose of the full adder is to perform binary addition and carrying the output through multiple adders to produce a carry and sum output. This is used in calculators to be able to do complex calculations through binary addition. In order to understand how full adders

work, minterm or maxterm expansion is used to get the Boolean expression of the full adder, it then can be simplified to conclude the most efficient circuit.

Requirements

The requirement for this lab is to simulate a full adder given three inputs with multiple sum outputs and a carryout output. We also coded a schematic in Vivado using VHDL as well as to have a testbench of the waveforms to match the results from Multisim. Then we programmed an FPGA board to show the outputs that match the truth table for a three input system.

Prelab

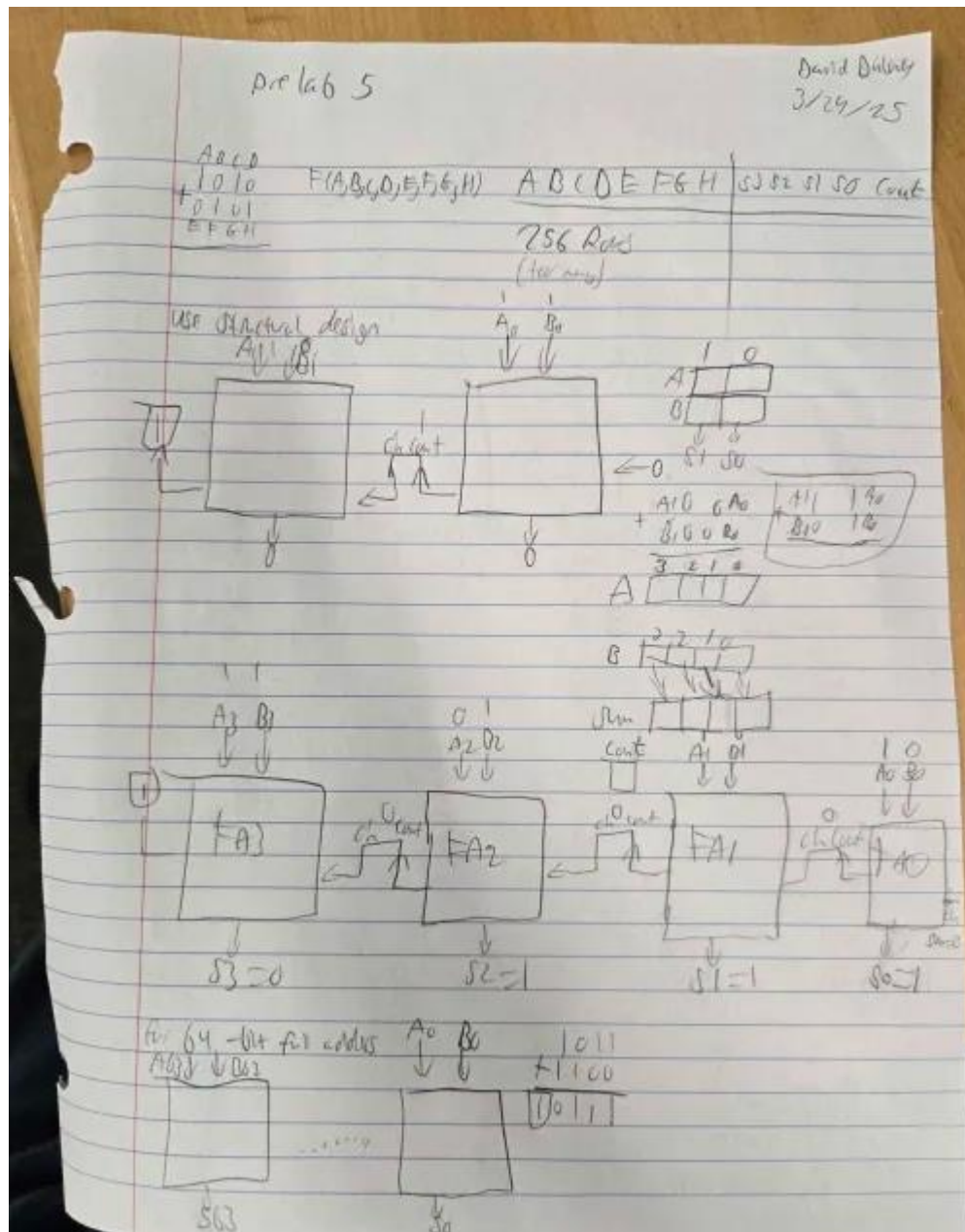


figure 1.1: prelab for full adder and four full adders connected to each other

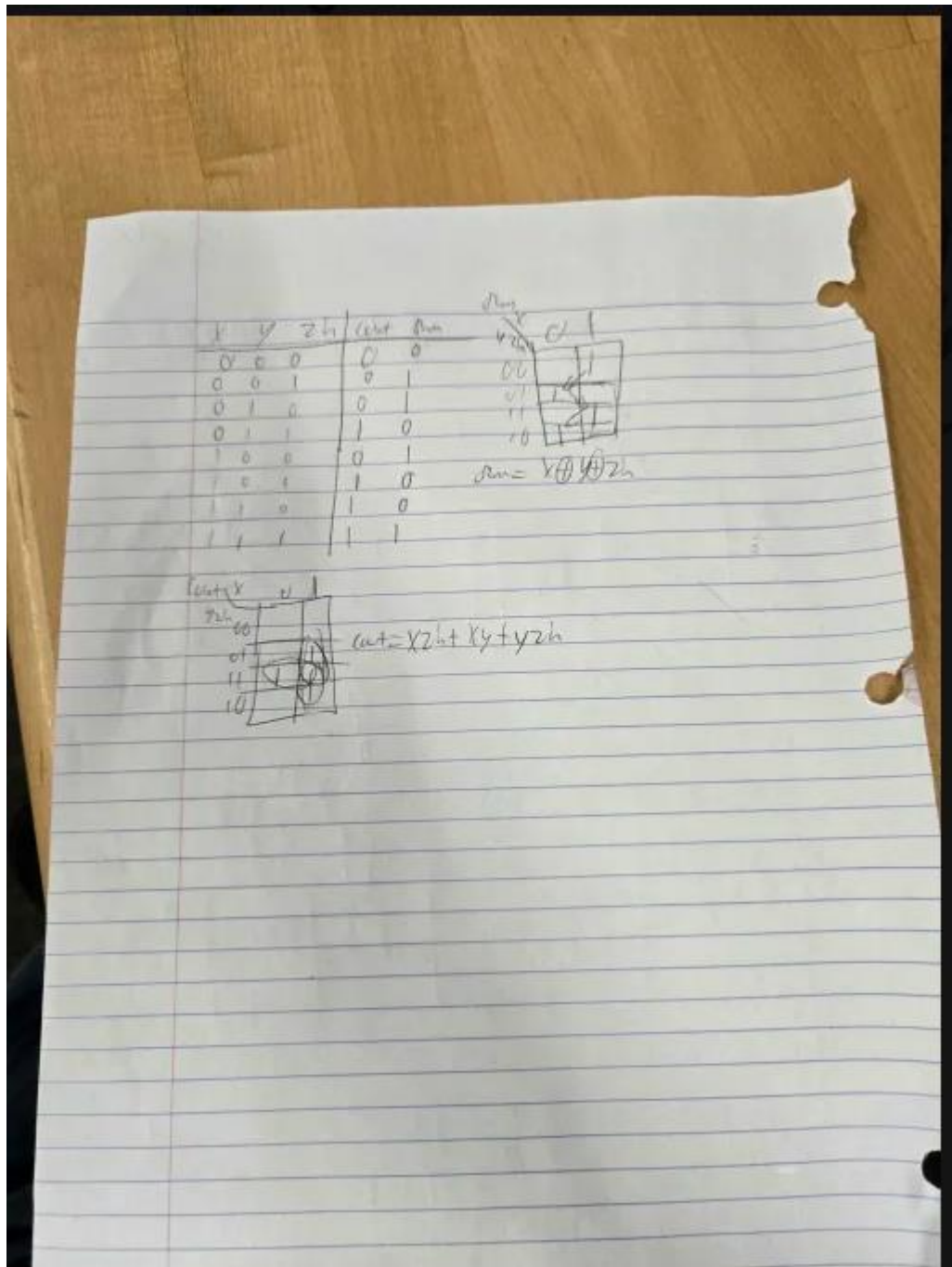


figure 1.2: Truth table of full adders with three inputs along with kmaps

Implementation

The first step in implementation was to build a testbench to show how a four adder system would behave given three inputs and 4 outputs. Then we programmed an FPGA board to behave the same as the behavioral simulation.

Design Code/Design Diagrams

DM code: for full adder

```
-----  
  
-- Company:  
  
-- Engineer:  
  
--  
  
-- Create Date: 03/24/2026 02:02:01 PM  
  
-- Design Name:  
  
-- Module Name: project_7_code - Behavioral  
  
-- Project Name:  
  
-- Target Devices:  
  
-- Tool Versions:  
  
-- Description:  
  
--  
  
-- Dependencies:  
  
--  
  
-- Revision:  
  
-- Revision 0.01 - File Created  
  
-- Additional Comments:  
  
--  
-----
```

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;


-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;


entity project_7_code is
    Port ( A : in STD_LOGIC_VECTOR (0 to 3);
          B : in STD_LOGIC_VECTOR (0 to 3);
          cin : in STD_LOGIC;
          sum : out STD_LOGIC_VECTOR (0 to 3);
          cout : out STD_LOGIC);
end project_7_code;


architecture Behavioral of project_7_code is
    component Project_7_Full_Adder is
        Port ( X : in STD_LOGIC;
              Y : in STD_LOGIC;
              Zin : in STD_LOGIC;

```

```

        Addition : out STD_LOGIC;

        InternalCout : out STD_LOGIC);

end component;

--internalcarryoutoffulladder0tointernalcarryinoffulladder1

signal wCoutfa0:std_logic;

--internalcarryoutoffulladder0tointernalcarryinoffulladder2

signal wCoutfa1:std_logic;

signal wCoutfa2:std_logic;


begin

FA0: Project_7_Full_Adder port map(x =>A(3), y => b(3), zin =>cin, addition =>
sum(3),internalcout => wcoutfa0);

FA1: Project_7_Full_Adder port map(x =>A(2), y => b(2), zin =>wcoutfa0, addition =>
sum(2),internalcout => wcoutfa1);

FA2: Project_7_Full_Adder port map(x =>A(1), y => b(1), zin =>wcoutfa1, addition =>
sum(1),internalcout => wcoutfa2);

FA3: Project_7_Full_Adder port map(x =>A(0), y => b(0), zin =>wcoutfa2, addition =>
sum(0),internalcout => cout);


end Behavioral;

```


Schematics/Implemented design

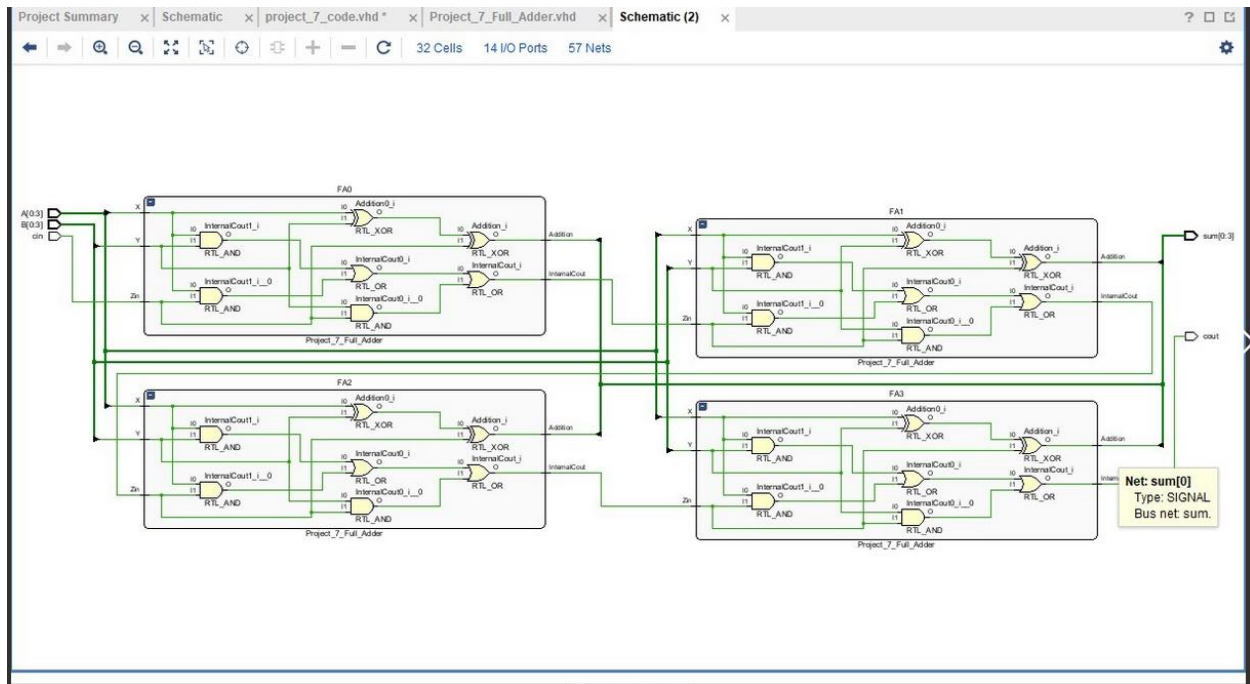


Figure 2.1: Schematic of expanded view of four full adders connected together

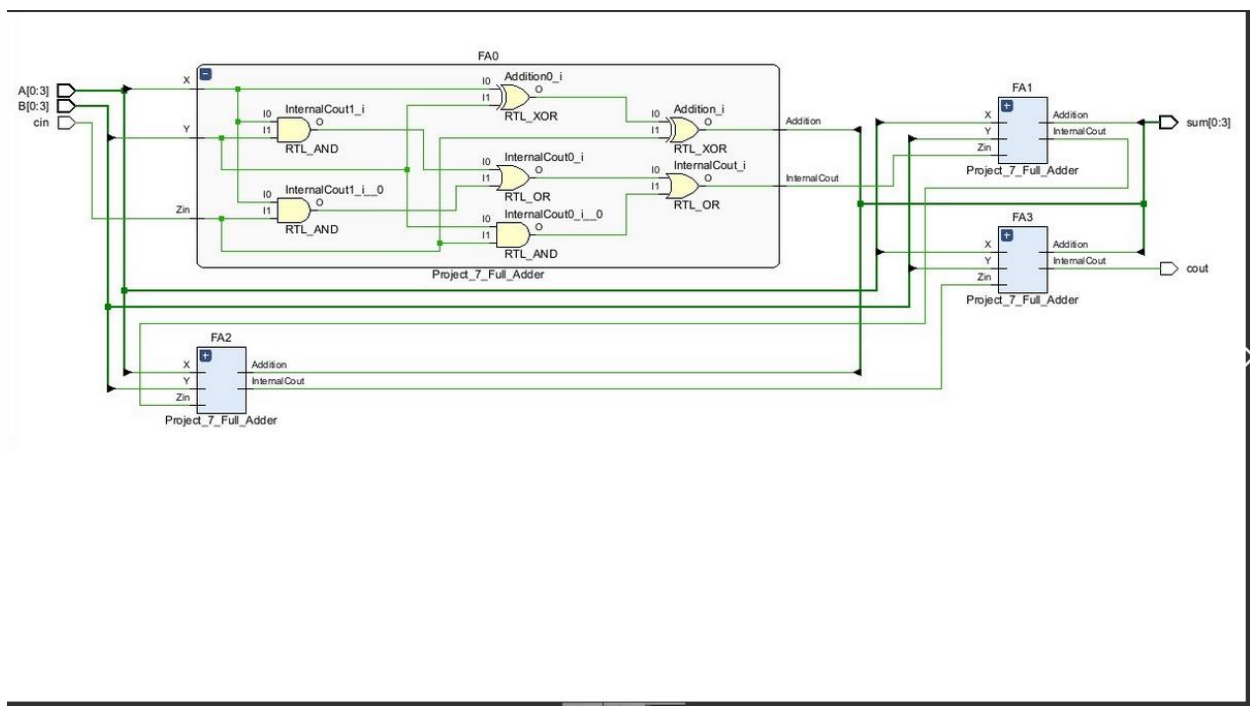


Figure 2.2: Schematic of the one expanded full adder and three full adders not expanded connected together

Testbench

TB code for full adder:

-- Company:

-- Engineer:

--

-- Create Date: 03/24/2026 03:07:15 PM

-- Design Name:

-- Module Name: Project_7_TB - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

--

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Project_7_TB is
-- Port ( );
end Project_7_TB;

architecture Behavioral of Project_7_TB is

component project_7_code is
    Port ( A : in STD_LOGIC_VECTOR (0 to 3);
          B : in STD_LOGIC_VECTOR (0 to 3);
          cin : in STD_LOGIC;
          sum : out STD_LOGIC_VECTOR (0 to 3);
          cout : out STD_LOGIC);
end component;

signal w4cin,w4cout: std_logic;
signal w4A, w4B, w4sum: std_logic_vector (0 to 3);

```

```

begin

-- assigning ports to wires

test: project_7_code port map(A => w4A, B => w4B, sum => w4sum, cin => w4cin, cout =>
w4cout);

process

begin

--adding binary inputs from 1-F

w4a <= "1011"; w4b <= x"C"; w4cin <= '0';wait for 10ns;

w4a <= x"A"; w4b <= x"E"; w4cin <= '0';wait for 10ns;

w4a <= x"8"; w4b <= x"9"; w4cin <= '0';wait for 10ns;

w4a <= x"4"; w4b <= x"7"; w4cin <= '0';wait for 10ns;

w4a <= x"D"; w4b <= x"E"; w4cin <= '0';wait for 10ns;

w4a <= x"8"; w4b <= x"3"; w4cin <= '0';wait for 10ns;

w4a <= x"2"; w4b <= x"4"; w4cin <= '0';wait for 10ns;

w4a <= x"6"; w4b <= x"8"; w4cin <= '0';wait for 10ns;

w4a <= x"A"; w4b <= x"F"; w4cin <= '0';wait for 10ns;

w4a <= x"E"; w4b <= x"C"; w4cin <= '0';wait for 10ns;

w4a <= x"1"; w4b <= x"9"; w4cin <= '0';wait for 10ns;

w4a <= x"7"; w4b <= x"3"; w4cin <= '0';wait for 10ns;

w4a <= x"B"; w4b <= x"C"; w4cin <= '0';wait for 10ns;

w4a <= x"C"; w4b <= x"4"; w4cin <= '0';wait for 10ns;

w4a <= x"D"; w4b <= x"7"; w4cin <= '0';wait for 10ns;

w4a <= x"F"; w4b <= x"9"; w4cin <= '0';wait for 10ns;

end process;

```

end Behavioral;

ARDUINO BOARD code for full adder:

-- assigning 9 ports as inputs

set_property PACKAGE_PIN V13 [get_ports {A[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]

set_property PACKAGE_PIN V14 [get_ports {A[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]

set_property PACKAGE_PIN U14 [get_ports {A[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]

set_property PACKAGE_PIN U15 [get_ports {A[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {1[3]}]

set_property PACKAGE_PIN W18 [get_ports {B[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]

set_property PACKAGE_PIN V19 [get_ports {B[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]

set_property PACKAGE_PIN U19 [get_ports {B[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]

set_property PACKAGE_PIN E19 [get_ports {B[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]

set_property PACKAGE_PIN U16 [get_ports {Cin}]

set_property IOSTANDARD LVCMOS33 [get_ports {Cin}]

--assigning 4 ports as outputs

set_property PACKAGE_PIN L1 [get_ports {Sum[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {Sum[0]}]

set_property PACKAGE_PIN P1 [get_ports {Sum[1]}]

```

set_property IOSTANDARD LVCMOS33 [get_ports {Sum[1]}]
set_property PACKAGE_PIN N3 [get_ports {Sum[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Sum[2]}]
set_property PACKAGE_PIN P3 [get_ports {Sum[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Sum[3]}]
set_property PACKAGE_PIN U3 [get_ports {Cout}]
set_property IOSTANDARD LVCMOS33 [get_ports {Cout}]

```

Implemented design for full adder:

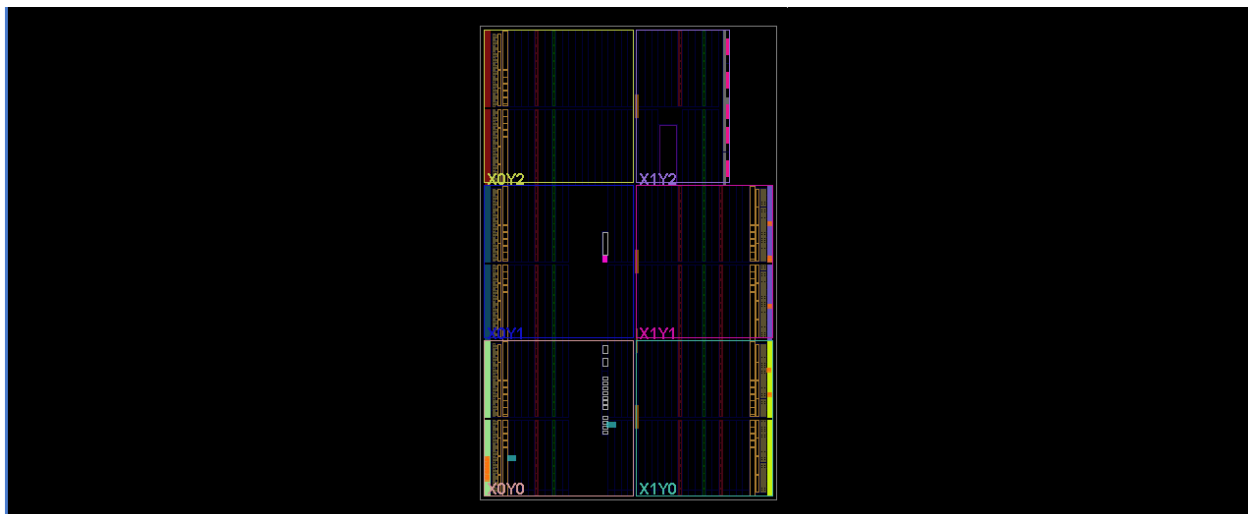


Figure 3: implemented design of four full adders connected for binary addition.

Waveform/Results

The behavioral simulation in figure 4 is showing the output of a three input system looks like with four full adders connected. For example, the hex value of B is 11 and the hex value of C is 12, add them together in normal math it's 23. In binary, which is why cout shows a 1 and sum shows a 7 in the first column.

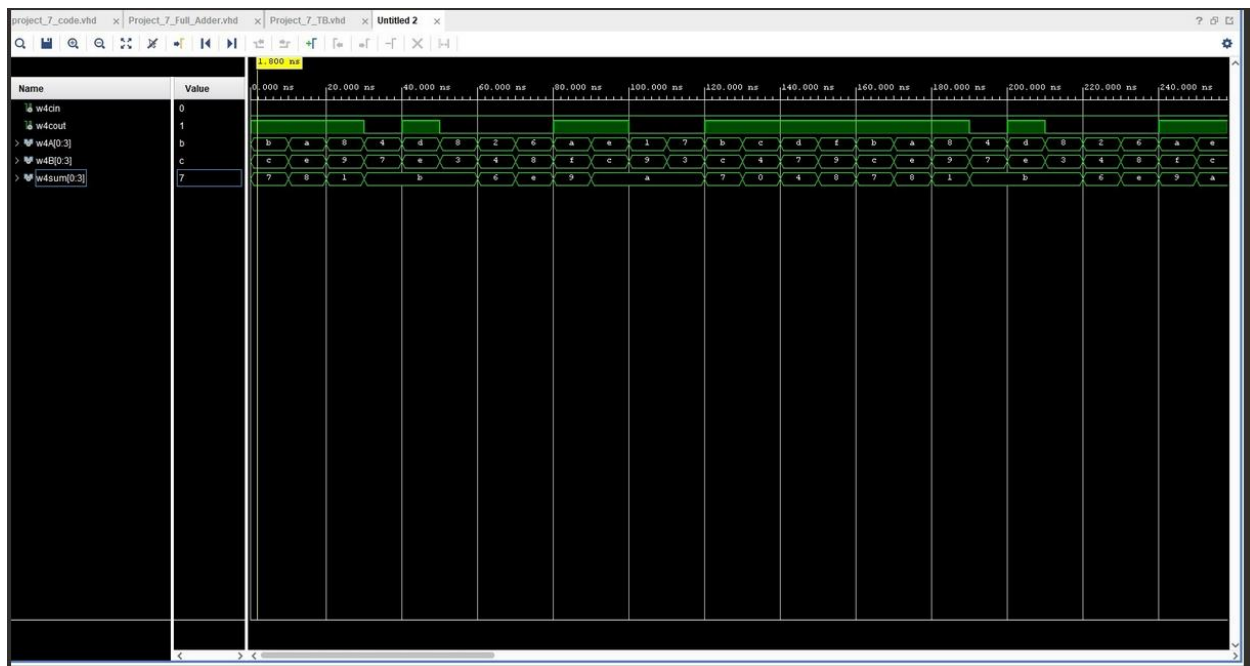


Figure 4: Behavioral simulation of four full adders doing binary addition

The result in figure 5.1 shows the Arduino board four adders doing binary addition inputs of “A” and “E” with the output being 1010 with a carryout of 1.

The result in figure 5.2 shows the Arduino board four adders doing binary addition inputs of “4” and “7” with the output being 1011 with a carryout of 0.

The result in figure 5.3 shows the Arduino board four adders doing binary addition inputs of “8” and “3” with the output being 1011 with a carryout of 0.

The result in figure 5.4 shows the Arduino board four adders doing binary addition inputs of “6” and “8” with the output being 1110 with a carryout of 0.

The result in figure 5.5 shows the Arduino board four adders doing binary addition inputs of “E” and “C” with the output being 1011 with a carryout of 1.

The result in figure 5.6 shows the Arduino board four adders doing binary addition inputs of “7” and “3” with the output being 1010 with a carryout of 0.

The result in figure 5.7 shows the Arduino board four adders doing binary addition inputs of “C” and “4” with the output being 0000 with a carryout of 1.

The result in figure 5.8 shows the Arduino board four adders doing binary addition inputs of “F” and “9” with the output being 1000 with a carryout of 1.

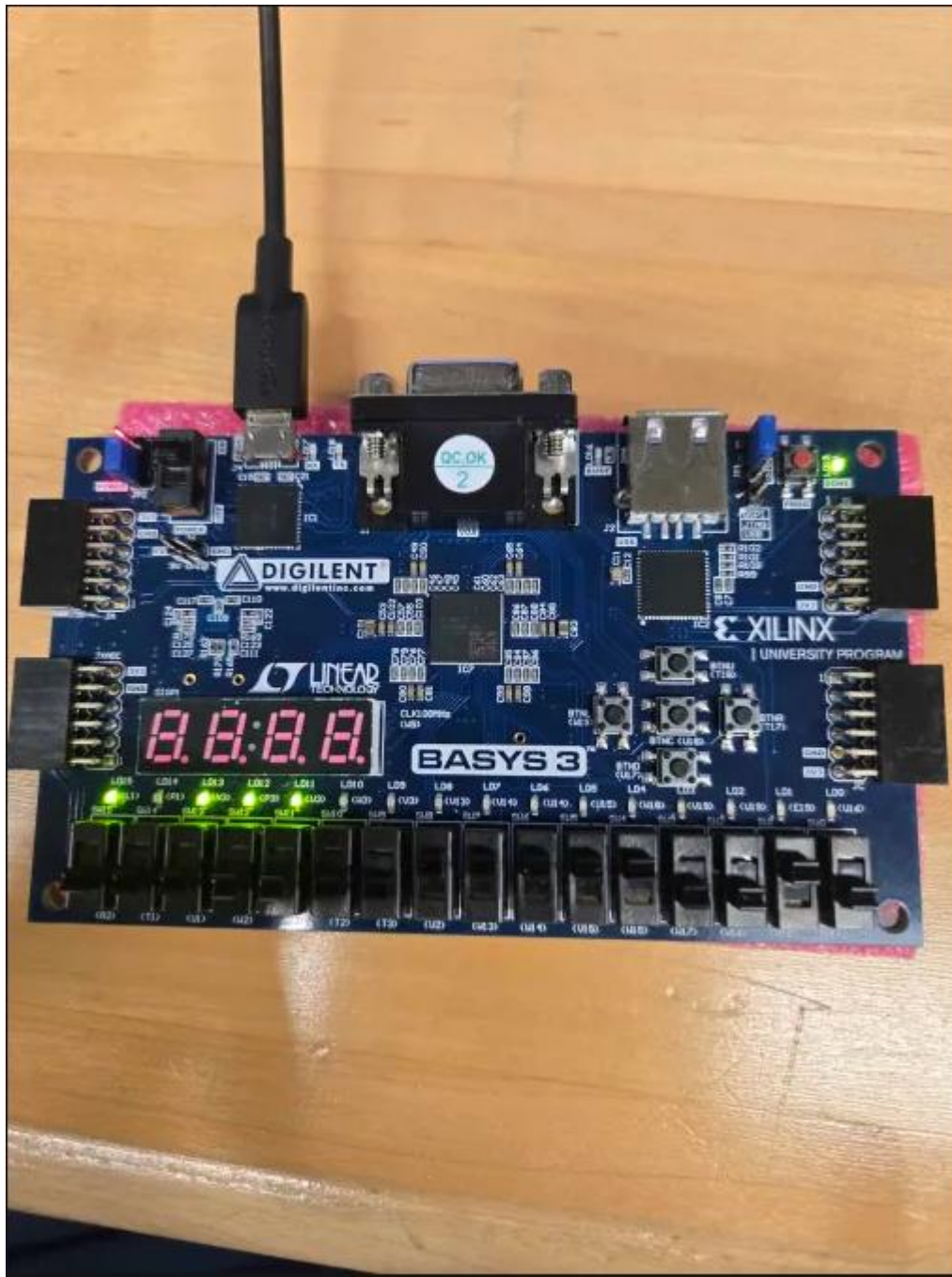


Figure 5.1: Arduino board for four adder binary addition of "A" and "E"

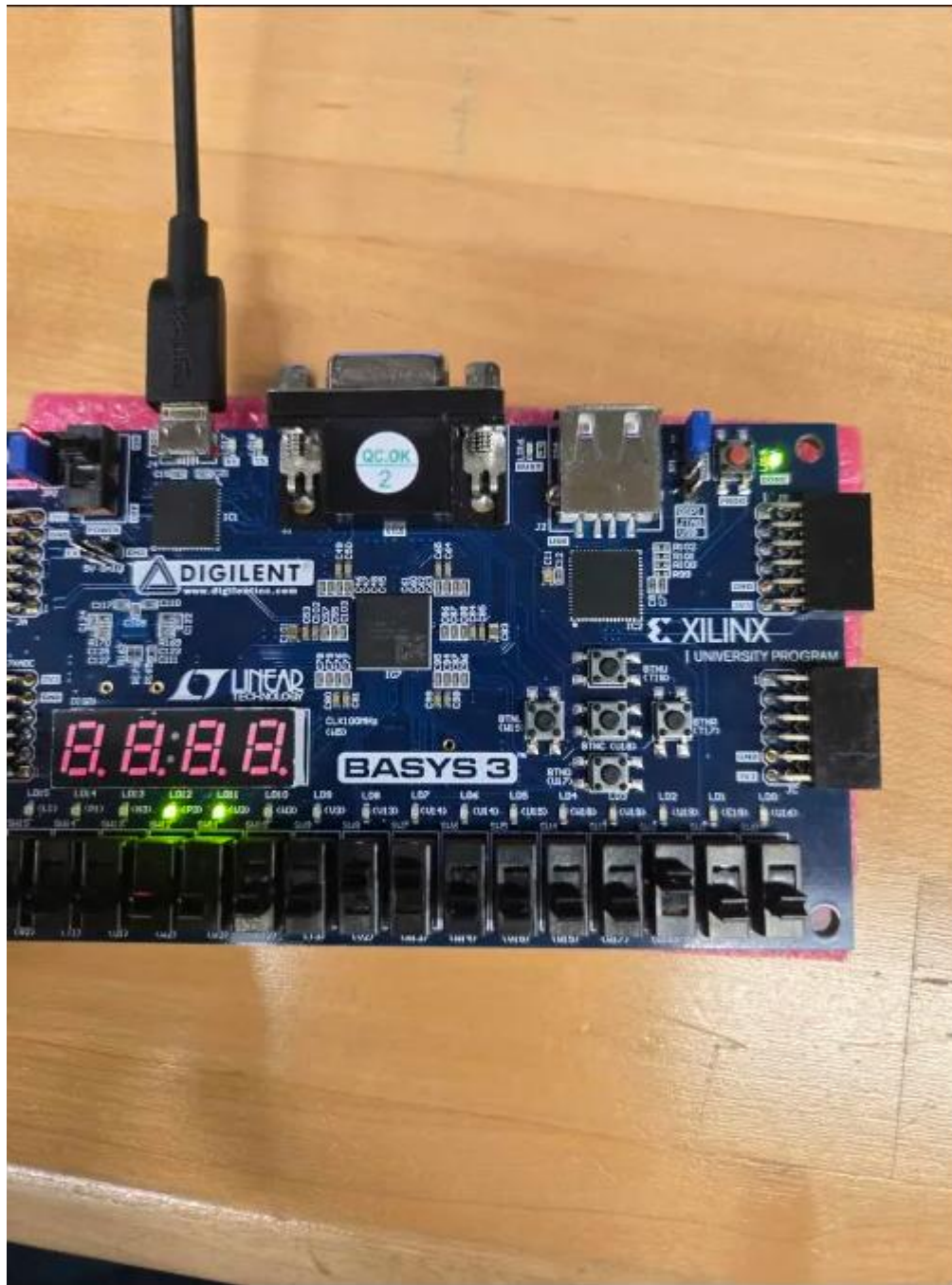


Figure 5.2: Arduino board for four adder binary addition of “4” and “7”

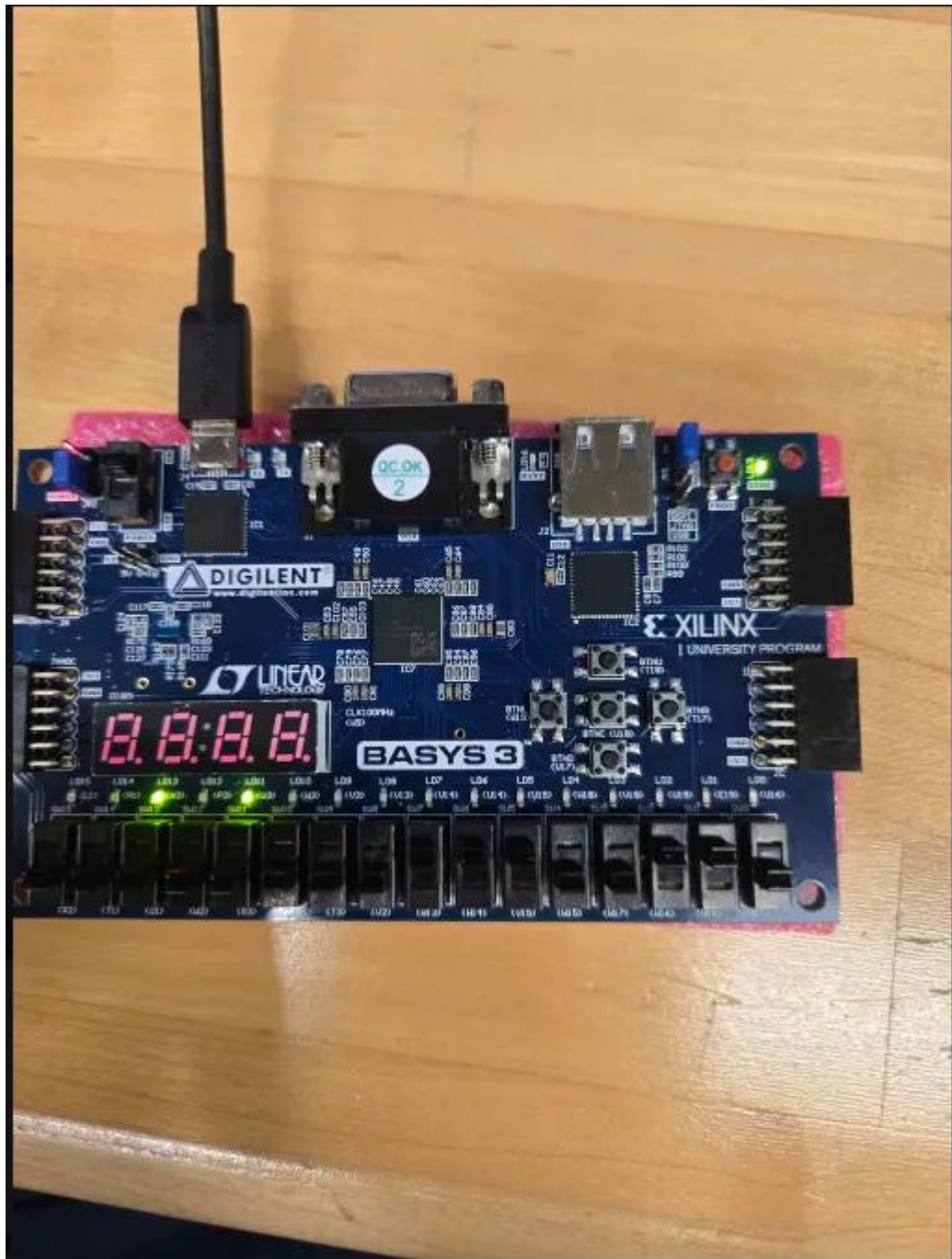


Figure 5.3: Arduino board for four adder binary addition of “8” and “3”

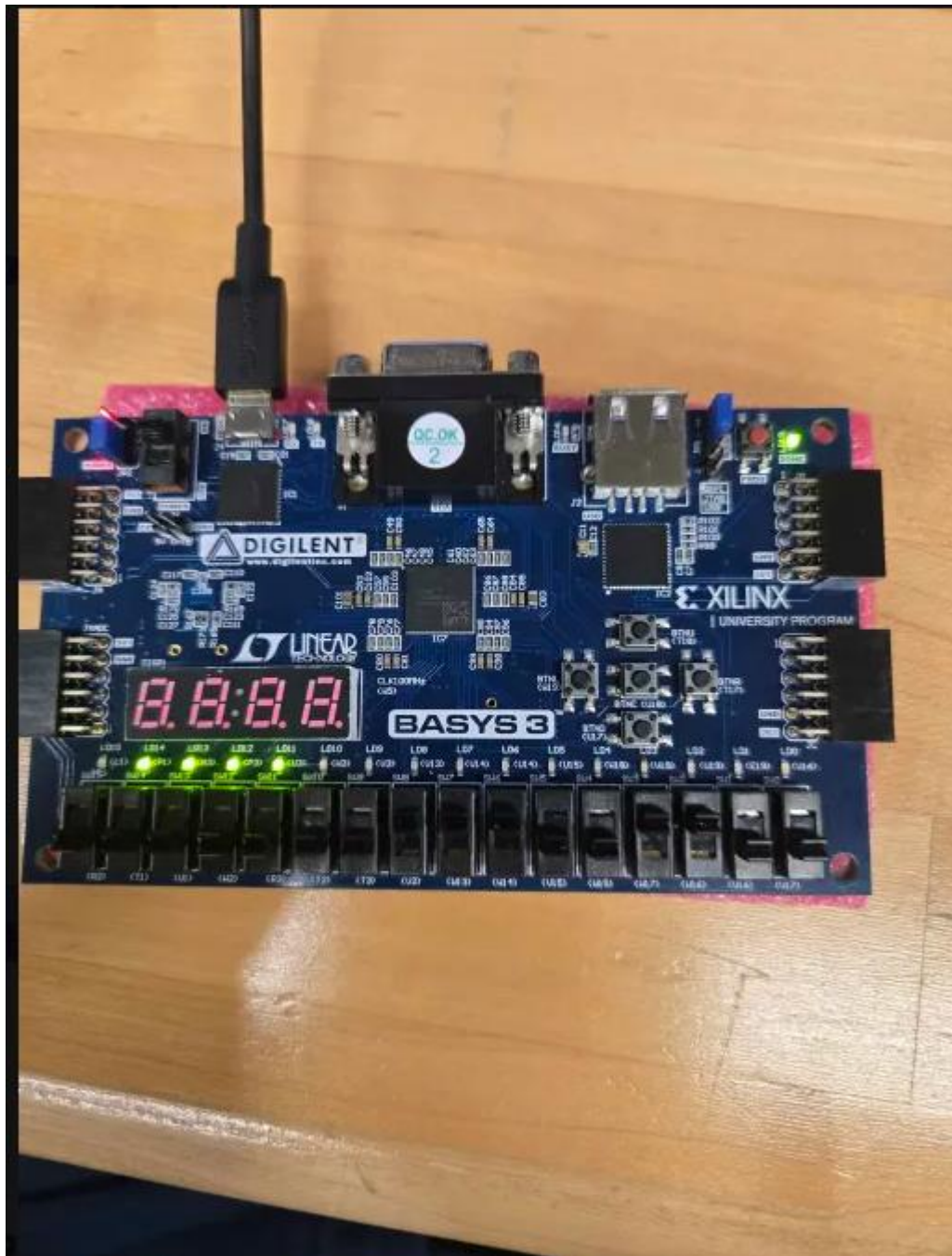


Figure 5.4: Arduino board for four adder binary addition of “6” and “8”



Figure 5.5: Arduino board for four adder binary addition of “E” and “C”

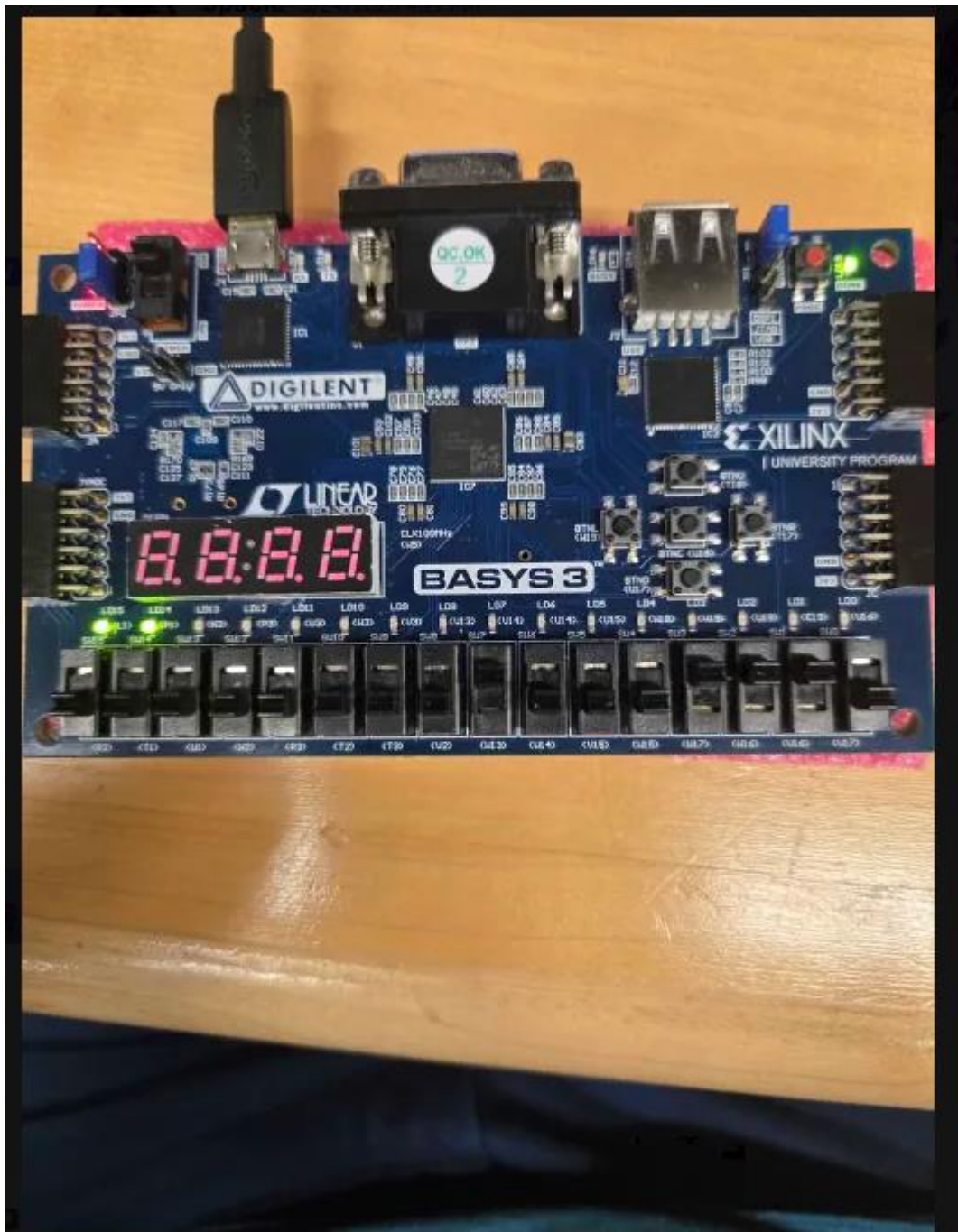


Figure 5.6: Arduino board for four adder binary addition of “7” and “3”

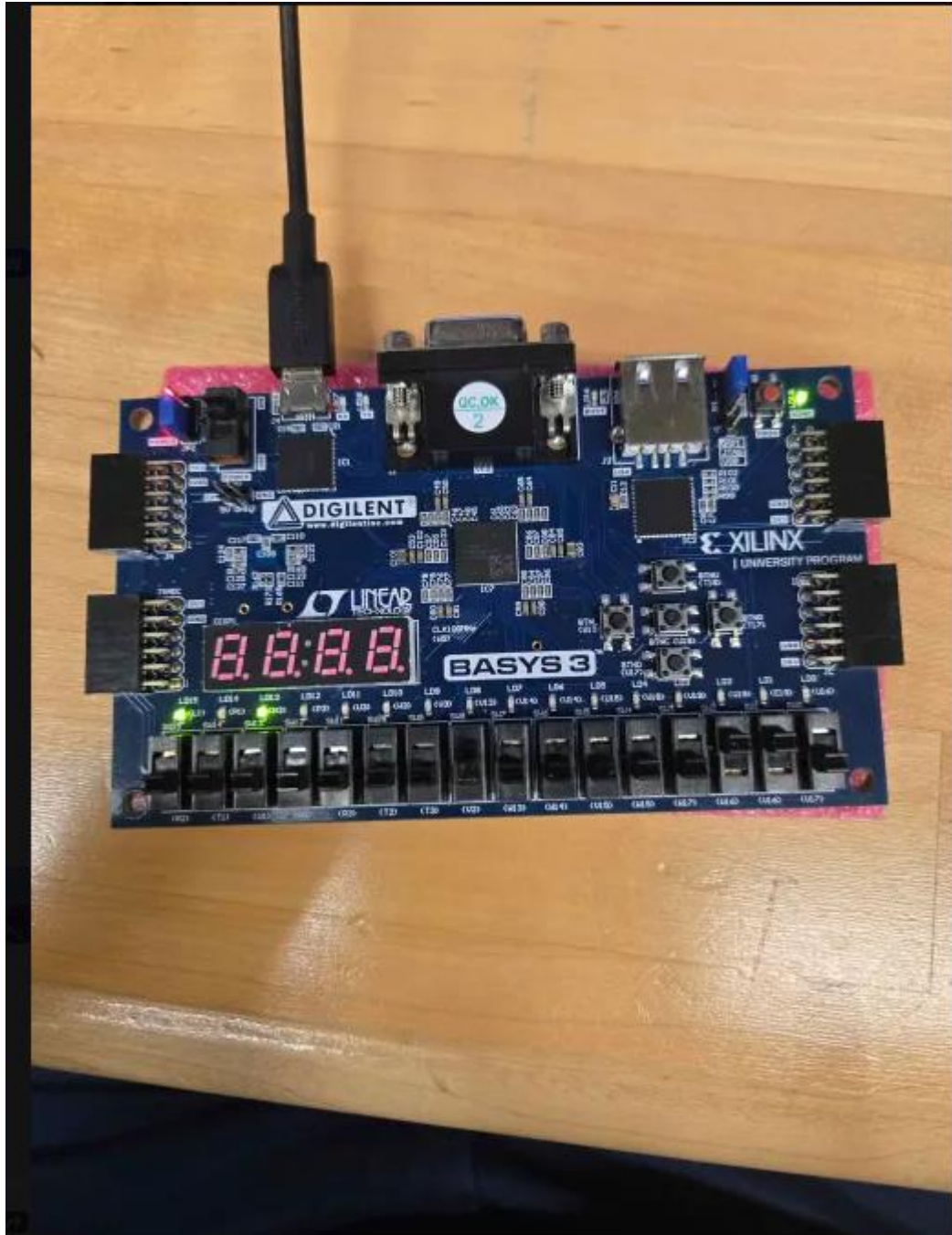


Figure 5.7: Arduino board for four adder binary addition of “C” and “4”

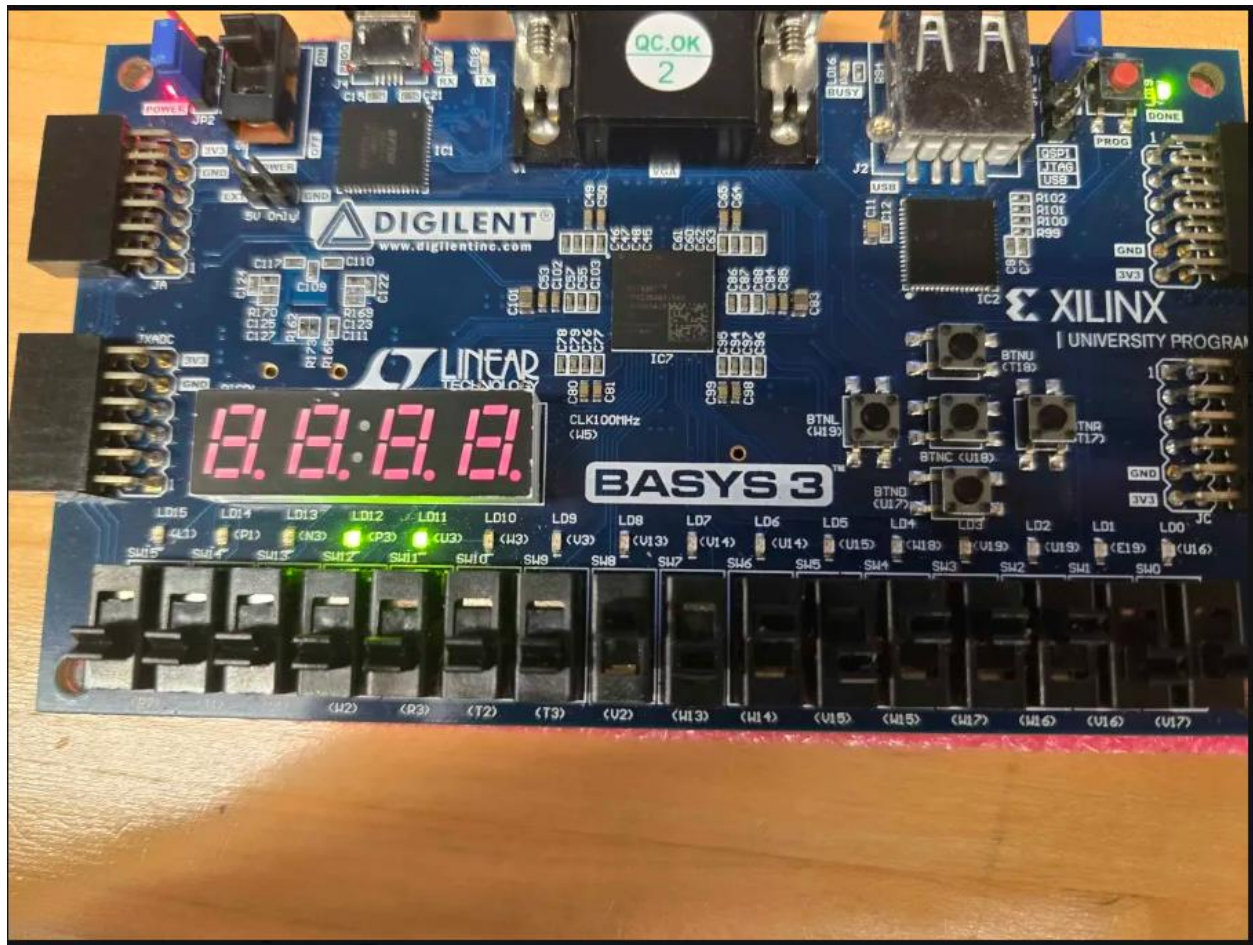


Figure 5.8: Arduino board for four adder binary addition of “F” and “9”

Conclusion

This project was designed to understand the binary addition four full adders compute when connected using a five-output system. The main takeaway is that full adders compute binary like normal, but even when there is a “0” for the carryout, that “0” gets sent to the next full adder’s addition of their binary column. This provides the adders a way to compute the next column whether a “0” or a “1” is in the carryout slot.